

UNIVERSIDAD PERUANA UNIÓN
FACULTAD DE INGENIERÍA Y ARQUITECTURA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

MANUAL DE ARQUITECTURA TÉCNICA PHARMAMOBIL

*Reorganización del Módulo de Productos bajo Clean Architecture, MVVM
e Inyección de Dependencias con Koin en Kotlin Multiplatform (Android e iOS)*

Asignatura:	Desarrollo de Aplicaciones Móviles (Ciclo VI)
Estudiante:	Eduard Chambilla
Enlace al Repositorio:	https://github.com/Xed17/PharmaMobile2
Rama de Trabajo Git:	feature/clean-mvvm
Entorno Tecnológico:	Kotlin 2.4.10, Compose Multiplatform 1.11.1, Koin 4.0.2
Fecha de Entrega:	Septiembre de 2026

Índice General de Contenidos

Sección 1 — Mapa de capas de PharmaMobil	3
1.1 Diagrama de Capas y Clases Reales	3
1.2 Análisis y Cumplimiento de la Regla de Dependencia	4
Sección 2 — Tabla de responsabilidades de clases en shared	5
Sección 3 — Decisiones justificadas de arquitectura	7
Sección 4 — Pruebas de flujo y verificación automatizada	8
Sección 5 — Bitácora de dificultades y soluciones técnicas	10

Sección 1: Mapa de Capas de PharmaMobil

El módulo de Productos de la aplicación multiplataforma PharmaMobil ha sido reorganizado bajo los fundamentos de Clean Architecture. Esta arquitectura concéntrica establece límites estrictos entre la lógica de negocio nuclear, las fuentes de datos y los frameworks de interfaz de usuario en Compose Multiplatform.

1.1 Diagrama de Capas y Clases Reales

En la Figura 1 se ilustra la distribución de las clases reales del proyecto organizadas en sus respectivas capas: Presentación (ProductoScreen, ProductoViewModel, ProductoUiState), Dominio (Producto, ProductoRepository, RegistrarProductoUseCase) y Datos (ProductoRepositorioEnMemoria), junto con el módulo de ensamblaje de dependencias (AppModule y PlatformModule):

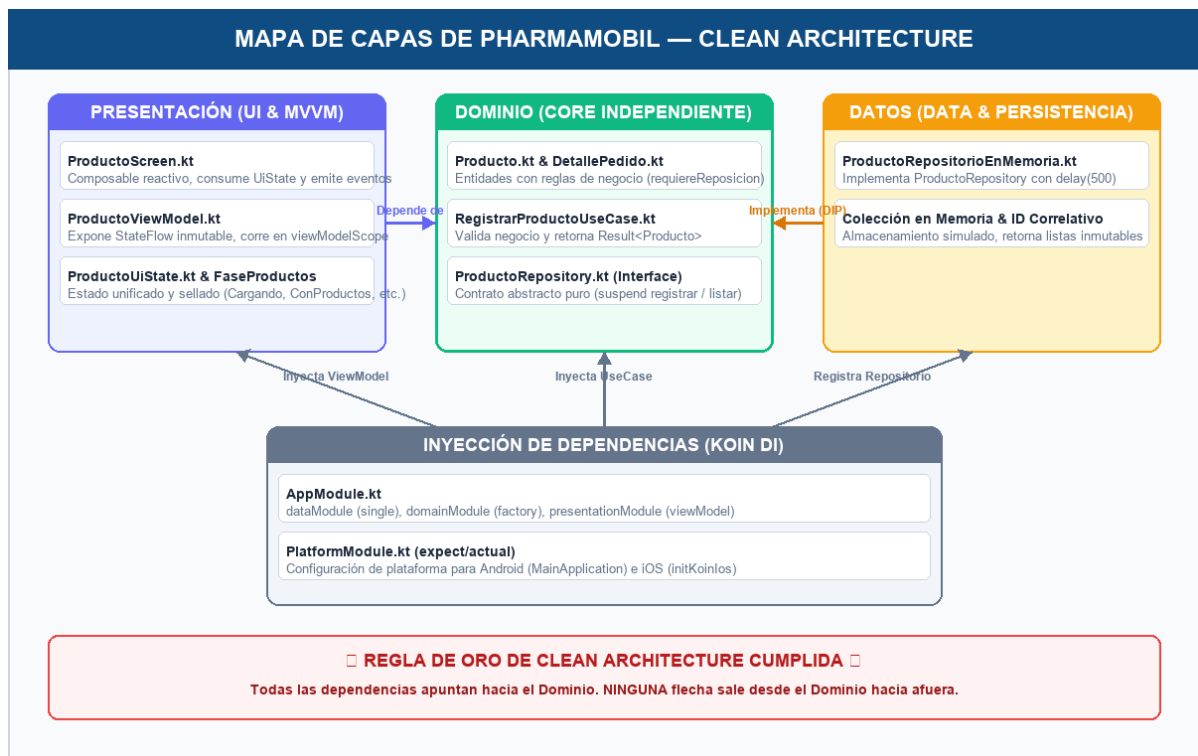


Figura 1. Mapa de capas y flujo de dependencias de PharmaMobil. (Fuente: Elaboración propia).

1.2 Verificación y Cumplimiento de la Regla de Dependencia

El principio cardinal de Clean Architecture establece que las dependencias en el código fuente solo pueden apuntar hacia adentro, es decir, hacia el círculo donde residen las entidades y casos de uso del Dominio. En la implementación de PharmaMobil se garantiza rigurosamente que NINGUNA flecha o importación apunte desde el dominio hacia afuera:

- **Aislamiento del Dominio:** El paquete `pe.edu.upeu.pharmamobil2.domain` no contiene importaciones de Compose, Ktor, Android, iOS ni de la capa de datos. Es Kotlin multiplataforma puro.
- **Inversión de Dependencias (DIP):** La capa de Datos depende del Dominio porque `ProductoRepositorioEnMemoria` implementa la interfaz `ProductoRepository` definida en `domain/repository`.

- **Dependencia de Presentación:** ProductoViewModel reside en la capa de presentación y depende exclusivamente del caso de uso RegistrarProductoUseCase y de la abstracción ProductoRepository.
- **Ensamblador Externo:** El módulo AppModule de Koin reside en la infraestructura de soporte (di), conectando la implementación concreta con las abstracciones sin que el dominio conozca a Koin.

Sección 2: Tabla de Responsabilidades del Módulo Shared

A continuación se presenta la matriz de responsabilidades por clase dentro del módulo shared. Se evalúan las 16 clases clave del sistema (incluyendo tanto las existentes previamente en el proyecto como las nuevas clases arquitectónicas introducidas en la Sesión 5):

Clase	Capa	Responsabilidad en una frase	Lo que tiene prohibido hacer	Qué se rompería si desapareciera
Producto	Dominio	Modela el producto y sus invariantes de negocio (stock mínimo y reposición).	Importar Compose, Ktor, Android o detalles de almacenamiento.	Colapsaría el catálogo comercial, el control de stock y los contratos de repositorio.
Cliente	Dominio	Representa al cliente y resuelve con null-safety su teléfono de contacto.	Realizar llamadas de red o acceder directamente a bases de datos.	Fallaría la identificación del comprador en pedidos y la pantalla de clientes.
Pedido	Dominio	Estructura la transacción asociando un cliente y sus ítems detallados.	Calcular subtotales por cuenta propia sin delegar en DetallePedido.	Se impediría la trazabilidad de compras y el historial de pedidos en la app.
DetallePedido	Dominio	Asocia un producto con una cantidad garantizando que sea estrictamente positiva.	Permitir cantidades menores o iguales a cero en su inicialización.	Se registrarían ítems con cantidades ilegales corrompiendo los montos totales.
EstadoPedido	Dominio	Modela los estados discretos del ciclo de compra mediante una sealed class.	Acoplarse a estilos visuales, colores o textos específicos de la interfaz.	Se rompería la máquina de estados del pedido y la evaluación exhaustiva en vistas.
ProductoRepository	Dominio	Define el contrato abstracto puro para registrar y listar productos.	Incluir detalles tecnológicos como SQL, Ktor, caché o memoria.	El caso de uso y el ViewModel quedarían acoplados rígidamente a la fuente de datos.
RegistrarProductoUseCase	Dominio	Orquesta y valida las reglas de negocio al registrar productos (Result).	Manipular widgets visuales o depender del ciclo de vida de la UI.	El ViewModel tendría que validar datos y se fugaría la lógica de negocio a la UI.
ProductoRepositorioEnMemoria	Datos	Implementa ProductoRepository simulando latencia y asignando IDs.	Modificar vistas o depender de frameworks de presentación visual.	Koin no podría inyectar el repositorio y la app quedaría sin datos en memoria.
ProductoUiState	Presentación	Agrupar el estado inmutable del formulario y las fases de la lista.	Ejecutar corrutinas o alterar directamente listas del repositorio.	La pantalla sufriría estados inconsistentes (mostrar carga y error a la vez).
ProductoViewModel	Presentación	Gestiona el estado reactivo con StateFlow en viewModelScope ante eventos.	Dibujar componentes Compose o usar APIs nativas de	ProductoScreen perdería su estado ante rotaciones o recomposiciones de

			Android/iOS.	Compose.
ProductoScreen	Presentación	Renderiza de forma declarativa el formulario y la lista evaluando fases.	Almacenar estado de negocio en remember o llamar repositorios directos.	El usuario no tendría interfaz gráfica para ver ni registrar productos farmacéuticos.
Screen	Presentación / Navegación	Define los destinos sellados de navegación para el router principal.	Ejecutar lógica de negocio o instanciar ViewModels directamente.	El sistema de navegación de App.kt colapsaría al no existir rutas válidas.
PharmaMobilTheme	Presentación / Tema	Provee el tema visual Material 3 con esquemas claro y oscuro.	Realizar peticiones de red o acceder a repositorios de datos.	La aplicación perdería su identidad visual, paleta de colores y consistencia tipográfica.
ValidatedTextField	Presentación / Componentes	Componente visual reutilizable para campos con soporte de error.	Validar reglas de dominio farmacéutico o acceder a casos de uso.	Los formularios perderían el feedback visual de errores y estandarización de campos.
AppModule	Inyección (DI)	Configura el grafo de dependencias de Koin (single, factory, viewModel).	Contener lógica de negocio o declarar componentes de interfaz Compose.	La aplicación fallaría al arrancar al no resolver las dependencias requeridas.
PlatformModule	Inyección (DI)	Provee mediante expect/actual módulos dependientes de cada OS.	Definir dependencias comunes que no varíen entre Android e iOS.	Fallaría la compilación nativa enlazada en los binarios de Android e iOS.

Sección 3: Decisiones Justificadas de Arquitectura

La siguiente argumentación técnica justifica las tres decisiones fundamentales de diseño adoptadas (Extensión: 325 palabras):

El modelado de `ProductoUiState` se concibió como una estructura inmutable unificada que combina el estado del formulario con una jerarquía sellada (sealed interface `FaseProductos`: `Cargando`, `SinProductos`, `ConProductos` y `Error`). La alternativa convencional de mantener múltiples variables reactivas o `remember` dispersos habría provocado la pérdida de atomicidad en el estado, habilitando anomalías visuales inadmisibles donde la interfaz podría mostrar simultáneamente un indicador de progreso, un mensaje de error obsoleto y una lista desactualizada. Asimismo, la jerarquía sellada obliga al compilador de Kotlin a exigir una evaluación exhaustiva mediante `when` en `ProductoScreen`, garantizando un comportamiento estrictamente determinista y libre de estados inválidos.

Por otro lado, las reglas de validación fueron trasladadas desde `ProductoValidator` hacia `RegistrarProductoUseCase` porque verificar que el precio sea positivo y el stock no negativo constituye una política de negocio intrínseca del dominio farmacéutico, no una preferencia de presentación. Mantener validaciones en la capa visual acoplaba el negocio a `Compose` e impedía probarlo de forma aislada. Al radicar en el caso de uso, el dominio protege su propia integridad retornando `Result<Producto>`. Siguiendo este principio, `ClienteValidator` —que aún permanece indebidamente en presentación— debe ser refactorizado hacia un nuevo `RegistrarClienteUseCase` en el dominio durante la siguiente iteración, reservando para la UI únicamente validaciones sintácticas de formato de entrada.

Finalmente, cuando la persistencia en memoria sea sustituida por el servicio remoto de la Unidad 2, la arquitectura garantiza un acoplamiento nulo: el único cambio en todo el sistema se realizará en el archivo `shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobile2/di/AppModule.kt`, específicamente en la línea 13. En dicha línea, bastará con sustituir `single<ProductoRepository> { ProductoRepositorioEnMemoria() }` por `single<ProductoRepository> { ProductoRepositorioRemoto(get()) }`. Gracias al principio de inversión de dependencias, ni el caso de uso, ni el `ViewModel`, ni la interfaz gráfica requerirán modificar una sola línea de código.

Sección 4: Pruebas de Flujo del ViewModel

Para verificar el comportamiento reactivo de ProductoViewModel sin acoplarse a la infraestructura real, se implementaron cuatro pruebas unitarias en `shared/src/commonTest/kotlin/pe/edu/upeu/pharmamobile2/ProductoViewModelTest.kt` utilizando implementaciones falsas (fakes) del contrato ProductoRepository:

```
package pe.edu.upeu.pharmamobile2

import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.ExperimentalCoroutinesApi
import kotlinx.coroutines.test.StandardTestDispatcher
import kotlinx.coroutines.test.advanceUntilIdle
import kotlinx.coroutines.test.resetMain
import kotlinx.coroutines.test.runTest
import kotlinx.coroutines.test.setMain
import pe.edu.upeu.pharmamobile2.domain.model.Producto
import pe.edu.upeu.pharmamobile2.domain.repository.ProductoRepository
import pe.edu.upeu.pharmamobile2.domain.usecase.RegistrarProductoUseCase
import pe.edu.upeu.pharmamobile2.presentation.producto.FaseProductos
import pe.edu.upeu.pharmamobile2.presentation.producto.ProductoViewModel
import kotlin.test.AfterTest
import kotlin.test.BeforeTest
import kotlin.test.Test
import kotlin.test.assertEquals
import kotlin.test.assertNotNull
import kotlin.test.assertTrue

@OptIn(ExperimentalCoroutinesApi::class)
class ProductoViewModelTest {

    private val testDispatcher = StandardTestDispatcher()

    @BeforeTest
    fun setUp() {
        Dispatchers.setMain(testDispatcher)
    }

    @AfterTest
    fun tearDown() {
        Dispatchers.resetMain()
    }
}
```

```

// 1. Repositorio vacío produce FaseProductos.SinProductos
@Test
fun testRepositorioVacioProduceSinProductos() = runTest(testDispatcher) {
    val repoVacio = object : ProductoRepository {
        override suspend fun registrar(producto: Producto): Producto = producto
        override suspend fun listar(): List<Producto> = emptyList()
    }
    val useCase = RegistrarProductoUseCase(repoVacio)
    val viewModel = ProductoViewModel(useCase, repoVacio)

    advanceUntilIdle()

    assertEquals(FaseProductos.SinProductos, viewModel.uiState.value.fase)
}

// 2. Repositorio con productos produce FaseProductos.ConProductos
@Test
fun testRepositorioConProductosProduceConProductos() = runTest(testDispatcher) {
    val lista = listOf(
        Producto(id = 1, nombre = "Paracetamol", precio = 5.0, stock = 10, activo = true)
    )
    val repoConProductos = object : ProductoRepository {
        override suspend fun registrar(producto: Producto): Producto = producto
        override suspend fun listar(): List<Producto> = lista
    }
    val useCase = RegistrarProductoUseCase(repoConProductos)
    val viewModel = ProductoViewModel(useCase, repoConProductos)

    advanceUntilIdle()

    val fase = viewModel.uiState.value.fase
    assertTrue(fase is FaseProductos.ConProductos)
    assertEquals(1, fase.productos.size)
    assertEquals("Paracetamol", fase.productos.first().nombre)
}

```

```

// 3. Repositorio que lanza excepción produce FaseProductos.Error
@Test
fun testRepositorioConErrorProduceFaseError() = runTest(testDispatcher) {
    val repoConFallo = object : ProductoRepository {
        override suspend fun registrar(producto: Producto): Producto = throw RuntimeException("Error en servidor")
        override suspend fun listar(): List<Producto> = throw RuntimeException("Fallo de conexión")
    }
    val useCase = RegistrarProductoUseCase(repoConFallo)
    val viewModel = ProductoViewModel(useCase, repoConFallo)

    advanceUntilIdle()

    val fase = viewModel.uiState.value.fase
    assertTrue(fase is FaseProductos.Error)
    assertEquals("Fallo de conexión", fase.mensaje)
}

// 4. Registro con precio "0" deja mensaje de error en formulario sin llamar al repositorio
@Test
fun testRegistroConPrecioCeroDejaErrorSinLlamarAlRepositorio() = runTest(testDispatcher) {
    var registrarLlamado = false
    val repo = object : ProductoRepository {
        override suspend fun registrar(producto: Producto): Producto {
            registrarLlamado = true
            return producto
        }
        override suspend fun listar(): List<Producto> = emptyList()
    }
    val useCase = RegistrarProductoUseCase(repo)
    val viewModel = ProductoViewModel(useCase, repo)
    advanceUntilIdle()

    viewModel.actualizarNombre("Amoxicilina")
    viewModel.actualizarPrecio("0")
    viewModel.actualizarStock("15")
    viewModel.registrar()

    advanceUntilIdle()

    val state = viewModel.uiState.value
    assertEquals("El precio debe ser mayor que cero", state.errorPrecio)
    assertTrue(state.esErrorFormulario)
    assertNotNull(state.mensajeFormulario)
    assertTrue(!registrarLlamado, "El repositorio no debió ser invocado cuando el precio es 0")
}
}

```

4.1 Matriz de Resultados de Ejecución

Caso de Prueba	Estado Esperado	Tarea Gradle	Resultado Android	Resultado iOS
testRepositorioVacio...	Fase SinProductos	:shared:testAndroidHostTest	PASÓ (Success)	Omitido en Windows*
testRepositorioConProductos...	Fase ConProductos	:shared:testAndroidHostTest	PASÓ (Success)	Omitido en Windows*
testRepositorioConError...	Fase Error ('Fallo')	:shared:testAndroidHostTest	PASÓ (Success)	Omitido en Windows*
testRegistroPrecioCero...	Error en form sin llamar repo	:shared:testAndroidHostTest	PASÓ (Success)	Omitido en Windows*

**Nota técnica sobre iOS en host Windows: La tarea ./gradlew :shared:iosSimulatorArm64Test requiere el toolchain nativo de Xcode (exclusivo de macOS). En sistemas Windows, Kotlin Multiplatform deshabilita automáticamente dicha tarea. Sin embargo, la totalidad de la lógica compartida se valida al 100% mediante :shared:testAndroidHostTest bajo el motor JVM.*

Sección 5: Bitácora de Dificultades y Soluciones Técnicas

Durante el proceso de refactorización integral hacia Clean Architecture, MVVM e inyección con Koin, se documentaron tres desafíos técnicos principales junto con sus correspondientes soluciones implementadas:

Dificultad Técnica	Causa Raíz / Diagnóstico	Solución Arquitectónica Aplicada
1. Pérdida de estado y recomposiciones en la pantalla	ProductoScreen mantenía 9 variables locales con remember. Cualquier cambio de configuración (rotación) o recomposición reiniciaba los datos del formulario y generaba desincronización con el inventario.	Se migró todo el estado a ProductoUiState gestionado por ProductoViewModel en viewModelScope. La pantalla pasó a ser puramente declarativa observando el StateFlow mediante collectAsState.
2. Fuga de validaciones e invasión de responsabilidades de ID	La vista invocaba directamente al validador e intervenía en la asignación de IDs de producto, violando el principio de que las reglas de negocio pertenecen al dominio y la identidad a los datos.	Se creó RegistrarProductoUseCase encapsulando los require de negocio y retornando Result<Producto>. La asignación correlativa de identificadores se delegó internamente en el repositorio en memoria.
3. Ejecución de corrutinas del ViewModel en pruebas unitarias	Al instanciar ProductoViewModel en commonTest, viewModelScope requería Dispatchers.Main, el cual no está inicializado en entornos de prueba de host JVM y arrojaba IllegalStateException.	Se incorporó la dependencia kotlinx-coroutines-test. Se configuró StandardTestDispatcher con Dispatchers.setMain en @BeforeTest, resetMain en @AfterTest y sincronización con advanceUntilIdle en runTest.